

Email and notifications

How a reservation event becomes an email: the two-table spine, the four templates, the dispatcher, and the transport seam.

This is the **implementation** guide. Two neighbours own adjacent subjects and are not repeated here:

- [Domain §9](#) — *who* is notified, as a business rule.
- [Operations §2, §4](#) — configuring SES, diagnosing mail that isn't arriving, running the sweeper.

1 The rule that decides the whole design

An email provider outage must not fail a booking.

The tempting implementation is one line inside `submitBooking`:

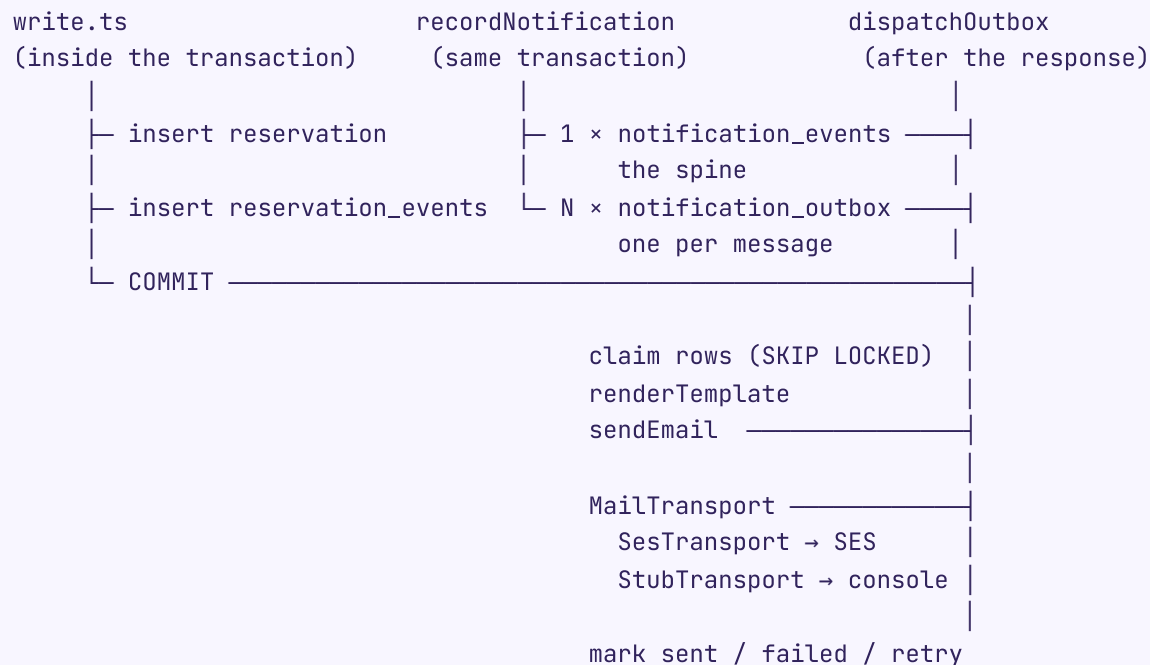
```
await sendBookingStatusChange(transport, actor.email, input); // don't
```

That puts an HTTP call to Amazon inside a database transaction. SES times out, the `await` rejects, the transaction rolls back, and a requestor who filled in a four-step wizard is told their booking failed — because a confirmation email didn't send. Moving the call after the commit only trades that for the opposite bug: the row exists and the email is lost with no record that it was owed.

So the send is **queued in the same transaction as the write, and delivered separately**. The queue row commits or rolls back with the reservation, atomically; delivery is a later, retryable step whose failure is visible on the row.

Everything below follows from that sentence.

2 The pipeline



Stage	File	Entry point
Trigger	<code>src/modules/reservations/write.ts</code>	4 call sites — <code>:285, :425, :757, :790</code>
Record	<code>src/modules/reservations/notify.ts</code>	<code>recordNotification</code> — <code>:88</code>
Payload	<code>src/modules/reservations/notify.ts</code>	<code>requestInformationFromDraft</code> <code>:143</code> , <code>loadRequestInformation</code> <code>:207</code>
Deliver	<code>src/modules/email/dispatch.ts</code>	<code>dispatchOutbox</code> — <code>:73</code>
Render	<code>src/modules/email/templates/index.ts</code>	<code>renderTemplate</code> — <code>:146</code>
Send	<code>src/modules/email/service.ts</code>	<code>sendEmail</code> — <code>:67</code>
Transpot	<code>src/modules/email/ses.ts, stub.ts</code>	<code>MailTransport</code> — <code>transport.ts:96</code>

One spine, two channels

In-app notifications and emails fire on the **same six transitions**. Recording them as two independent mechanisms would mean two sets of triggers in `write.ts`, drifting apart the first time an event is added. So one `notification_events` row is written per event and the channels fan out from it:

- `recipient_user_id` is the **in-app** channel.
- `notification_outbox` rows are the **email** channel — one row per message, so an event that mails the requestor and the admins separately is two rows pointing at one event.

The in-app channel is written but never read. Nothing in `src/` selects from `notification_events`. There is no bell, no feed, no `read_at` update path — only the two indexes the migration created for queries that don't exist yet. The rows are accumulating correctly and are ready for a UI; the UI is unbuilt. See §12.

3 The two tables

Migration: `src/modules/db/migrations/20260824130235_create_notification_tables.ts`

`notification_events` — the spine

Column	Notes
<code>reservation_id</code>	Nullable . The submitted digest is about a submission, not one row. ON DELETE CASCADE .
<code>event</code>	One of six, under <code>notification_events_event_check</code>
<code>recipient_user_id</code>	Nullable — an event whose only channel is email has no in-app recipient. FK to <code>users</code> , cascade.
<code>title, body, link</code>	Compact, for a bell list — <i>not</i> the email card. link is e.g. <code>/manage?ref=VR-1042</code> .
<code>read_at</code>	Nullable . Nothing sets it today.

All six events target exactly one user, which is why the in-app recipient is a column rather than a join table. An event needing several in-app recipients (a future admin digest) wants that join table; it was deliberately not built until one exists.

notification_outbox — the email channel

Column	Notes
<code>event_id</code>	NOT NULL , cascade. A queued message without the event that caused it is unexplainable.
<code>template</code>	The key into TEMPLATES
<code>recipient</code>	One address, or several comma-joined — the dispatcher splits them
<code>cc</code>	jsonb array. Read back for auditing, not searched; a delimited text column would force an address-escaping convention.
<code>payload</code>	jsonb . The template's input, pre-formatted at enqueue time — see §5.
<code>status</code>	pending sending sent failed
<code>attempts,</code> <code>next_attempt_at,</code> <code>last_error</code>	Retry state — see §8
<code>provider_message_id,</code> <code>sent_at</code>	SES MessageId and when

Two CHECK constraints and one index carry real weight:

- `notification_outbox_status_check` — the four statuses.
- `notification_outbox_sent_check` — `(status = 'sent') = (sent_at is not null and provider_message_id is not null)`. A sent row has both; an unsent row has neither. Half-written state is rejected by the database.
- `notification_outbox_due_idx` on `(next_attempt_at)` where `status = 'pending'` — the dispatcher's only query. Partial, because **sent** rows are the vast majority and never appear in it.

4 Events, templates, recipients

Six events, four templates. The mapping is not one-to-one, and both collapses are deliberate.

Event	Template	To	Cc
submitted	booking-submitted	Requestor	—
submitted	admin-new-request	Site admins (comma-joined)	—
approved	booking-status-change (status: "Approved")	Requestor	Site admins
rejected	booking-status-change (status: "Rejected")	Requestor	Site admins
cancelled	booking-status-change (status: "Cancelled")	Whoever did not act	Site admins, only when an admin cancelled
driver_assigned / driver_changed	driver-assignment	Requestor	Site admins

Approved, Rejected and Cancelled share one template because they differ only by a heading, a sentence, and whether there is a reason to quote. The status is a payload field, not a template name.

An assignment change is NOT a status change — the reservation stays Approved — so it has its own template rather than a fourth branch of `booking-status-change`. This is also why the `decision: null` path in `decideReservation` ([write.ts:747](#)) has to compare the old and new assignment on **both** sides: a legal field-only save is usually not news, but a change to who or what is running the trip is one of the six events the requestor is promised.

The stored event values keep the driver_* spelling even though a van change fires them too. They live in `notification_events.event` under a CHECK constraint; renaming them buys nothing a requestor ever sees. The *copy* was made neutral instead — "Van assignment set" / "Van assignment changed" — and names neither side.

One digest per submission, not one mail per trip

A `BookingDraft` carries `trips: TripDraft[]`, and `submitBooking` writes one reservation per trip, each with its own reference. So the `submitted` event is recorded **after** the trip loop ([write.ts:285](#)), still inside the same transaction, with `reservationId: null` and a payload carrying every reference. Enqueuing beside each inserted row would drop four near-identical mails into an inbox within the same second.

The decision mails stay **per reservation**: approval, rejection and cancellation are per-trip outcomes, and a submission's trips can and do diverge (two approved, one rejected). Grouping those would mean holding a mail back until every trip in the submission had been decided.

Because that one event has a null `reservation_id`, it does not inherit the `ON DELETE CASCADE` the others get. Acceptable today — reservations are never deleted, cancellation is a status — but a future hard-delete would strand these rows.

Recipients

One function decides, and it is the only place the rule lives: `adminRecipients` (`recipients.ts:76`), over the pure `selectAdminEmails` (`:46`). The rule itself — active, `notify`, site matches or `all`, name-sorted, acting party excluded, email-less rows dropped — is documented in [domain §9](#).

Two implementation notes that matter here:

- **The whole `admin_whitelist` table is read, then filtered in JS.** Not a `WHERE` clause. It holds eight rows, and the point is that the dev preview resolves its To/Cc from `ADMIN_WHITELIST_SEED` through the *same* function — a SQL filter the preview path could not share is how the two would diverge.
- **A message with nobody to send it to is skipped, not queued (`notify.ts:110`).** Writing it would manufacture a permanent `VALIDATION_FAILED` and train whoever reads the failure log to ignore it. The event still stands; only that channel is empty.

EMAIL_ALWAYS_CC

Applied in `sendEmail` (`service.ts:88`), **not** at any enqueue site — so a new template, a new event, or a new caller gets the standing copy without knowing it exists. Consequences worth knowing:

- It is **not** written to the outbox row's `cc`, which stays the record of who the *decision* copied.
- Deduped against both `to` and `cc`, case-insensitively (`extraCc`, `:30`). SES accepts the same address in both headers and delivers it twice, and the standing recipient is exactly the person most likely to also be a site admin.
- Validation runs on the **caller's** message, before the standing copy is appended. A misconfigured `EMAIL_ALWAYS_CC` cannot fail an otherwise valid notification.

5 The payload contract

Pre-formatted at enqueue time, snapshotted, never recomputed.

Two independent reasons, both load-bearing:

1. **An email has no timezone.** A `timestamp_tz` rendered in the wrong one shows the wrong day. `start_at` and `end_at` go through `instantInManila` before formatting (`notify.ts:258`) — the single most load-bearing line in that file. Eight hours out of every twenty-four, skipping it reads as the previous day.
2. **A dispatcher that re-read the reservation would lie.** It would send an "approved" notice describing a trip that has since been edited, or mail whoever the whitelist names *today* about a decision made last week.

Rules that follow:

- **Format with lib/tz.ts helpers only** — `formatPlainDate`, `formatPlainTime`, `formatPlainDateTime`, `instantInManila`. `tz.test.ts` has a drift guard that walks all of `src/` and fails if any other file formats a date or touches `Intl.DateTimeFormat`.
- **Empty values become EM_DASH (—), never ""**. A blank string renders as a label with nothing under it, and every payload string is `min(1)` — so a blank field would fail the row permanently. A missing value degrades to a dash instead of burning the row.
- **Templates receive display strings and never a Date**. The contract is "caller formats, template lays out".

Two builders produce the payload:

Function	Used by	Shape
<code>requestInformationFromDraft</code> (<code>notify.ts:143</code>)	<code>submitBooking</code>	1..N trips, from the draft in hand — nothing re-read
<code>loadRequestInformation</code> (<code>notify.ts:207</code>)	<code>decideReservation</code> , <code>cancelReservation</code>	Exactly one trip, from the stored row

`manageUrl` / `adminUrl` come from `env().APP_URL`.

The "Assigned van" block

`assignedVanOf` (`notify.ts:322`) fills it from whichever side holds the assignment — the roster row or the typed-in rental — with left joins on both. Three decisions:

- **Rental status is deliberately not disclosed**. The requestor is told the plate and a mobile so they can meet their van; who owns it is Admin Support's business, and the same block serves both cases.
- The block is **omitted entirely** only when neither side is assigned. One side without the other dashes the missing half rather than hiding the half that exists.

- It returns `RequestDriver` — a legacy name from before the fleet split, reused on purpose for a value that now names a van.

Payload validation is a schema, at render time

`templates/index.ts` pairs each template with a zod schema (:106), and `renderTemplate` (:146) validates before rendering. Validating *here* turns a shape mismatch into a permanent, visible failure on the row — rather than a renderer throwing on `undefined.map`, or worse, rendering an empty string that `sendEmail` then refuses as `VALIDATION_FAILED` with nothing saying which field was missing.

Two fields are deliberately loose, and both are **temporary**:

Field	Why	When to tighten
<code>tripDetails = text.catch(EM_DASH) (:42)</code>	<code>details</code> became required after some rows were already queued, and <code>INVALID_PAYLOAD</code> is permanent — strict validation would strand a pre-deploy row forever	Once no pre-deploy row can remain in the outbox
<code>driver.carType = text.optional() (:54)</code>	Same story, one field deeper	Same

Leaving these loose past their release is how a real missing field starts rendering as a dash instead of failing loudly.

6 Templates

Four templates, one envelope, one shared card stack.

Template	Audience	Subject
<code>booking-submitted</code> <code>booking-submitted.tsx</code>	Requestor	Van booking VR-... received — Pending approval, or ... — N trips, Pending approval
<code>admin-new-request</code> <code>admin-new-request.tsx</code>	Site admins	New van reservation — <requestor> (<site>), or ... (<site>), N trips
<code>booking-status-change</code> <code>booking-status-change.tsx</code>	Requestor	Van reservation VR-... — Approved / Rejected / Cancelled
<code>driver-assignment</code> <code>driver-assignment.tsx</code>	Requestor	Van reservation VR-... — van assignment set / changed

Every one is the same three parts:

1. **shell.tsx** — the envelope: document, preview text, page background, the white card. **preview** is the snippet clients show beside the subject; left unset it falls back to the first words of the body, which for these templates is "Hi <name>," — the least informative sentence in the mail.
2. **Its own copy** — heading, lead, and whatever the event needs quoted.
3. **request-information.tsx** — the booking itself, a card stack mirroring **step-review.tsx** so the mail reads like the screen the requestor just confirmed.

Subjects name the reference **only when there is exactly one**. Six references would push past the ~70 characters most clients show, truncating the part that identifies the mail; they are all in the body regardless. The admin subject leads with the requestor and site instead, because an admin triaging a full inbox sorts on those, not on a reference they have not seen yet.

RequestTrip is a **discriminated union on mode**, not one interface with everything optional (**request-information.tsx:63**). The two modes genuinely describe different things: a standby block has a window and a reporting point and no drop-off. Rendering it through pickup labels is how you get **Pickup time: 6:00 AM - 6:00 PM**. The branch lives there, once, so the four templates never repeat it.

Styles are inline hex, deliberately

templates/styles.ts holds one shared object. Inline because email clients strip **<style>** blocks and know nothing of cascade. **Literal hex, not the app's CSS custom properties** — **var()** does not resolve in Outlook, and a token that silently falls back to black is worse than a hardcoded colour someone has to update in one place.

This is the one place in the codebase where hardcoding a colour is correct. See [architecture §9](#) for the rule it is an exception to.

Adding a template

1. Write `templates/<name>.tsx`: compose `EmailShell` + `RequestInformation`, import the shared `styles`, and export `render<Name>` returning `{ subject, html, text }`. Both bodies are required — a text alternative keeps mail out of spam folders and readable in text-only clients.
2. Add a zod schema and register it in `TEMPLATES` (`templates/index.ts:106`). An unregistered name fails its row as `UNKNOWN_TEMPLATE` — permanent.
3. Add it to `NotificationTemplate` in `notify.ts:49`.
4. Add a fixture to `templates/fixtures.ts` (`PREVIEWS, :220`) — that is what the dev preview lists, and the cheapest way to look at both halves without sending anything.
5. Enqueue it from a `recordNotification` call in `write.ts`.

7 The transport seam

```
sendEmail (service.ts) → MailTransport → SesTransport | StubTransport
```

`MailTransport` (`transport.ts:96`) is the only surface the rest of the app depends on. Nothing upstream imports the AWS SDK — swapping providers means adding one class beside `SesTransport` and nothing else.

`from` is deliberately **not** on `EmailMessage`: the app sends as a single verified SES identity (`EMAIL_FROM`), so the sender is a property of the transport, not of each message.

Selection

`selectMailTransport(env())` (`select-transport.ts:22`), mirroring `selectAuthProvider`. `EMAIL_MODE=ses` builds SES; **anything else** builds the stub.

Returns `null` only when `sas` is selected without `AWS_REGION` or `EMAIL_FROM`. `env.ts`'s `superRefine` already refuses to boot in that state, so this is defence-in-depth for a caller that built an `Env` some other way. Callers treat `null` as "nothing to send through" and leave rows pending.

Credentials are not required. Absent keys are a *supported* configuration, not an incomplete one: the SES client then authenticates through the AWS SDK's default credential chain — a named profile, an SSO session, an instance or task role — each of which refreshes itself, unlike a pasted key. `sesClientConfig` (`ses.ts:131`) **omits** the `credentials` property rather than passing an empty object, because passing it at all opts *out* of that chain: an explicit-but-blank object authenticates as nobody.

The stub fails closed

`StubTransport`'s constructor throws unless `NODE_ENV` is in an **allowlist** of `development` / `test` (`stub.ts:15`). Deliberately an allowlist, not `nodeEnv` $\equiv$ `"production"` — a blocklist fails *open*, admitting the stub for `undefined`, `"`, a typo, or any future environment name. A stub in production silently drops every email, so the guard turns that data-loss into a loud boot failure.

It records every message in `sent` and logs one line per send:

```
[email:stub] → juan@example.com · cc: a@x, b@x · Van booking VR-1042 received (stub-1)
```

The `cc` is in that line because "who else got this?" is the question the log exists to answer, and it is invisible otherwise.

SES

`SesTransport` (`ses.ts:167`) makes one SESv2 `SendEmailCommand` call. The client is built once and captured, so a single connection pool is reused across sends. `sendCommand` is injectable so tests script SES responses without a network — the same pattern `CgsAuthProvider` uses for `fetch`.

`CcAddresses` is **omitted rather than sent empty**: SES accepts `[]`, but an empty header is noise in every recipient's client.

8 Failures and retries

Four failure codes, **Extracted** from `ErrorCode` (`transport.ts:56`) so removing one from `ERROR_STATUS` breaks compilation here rather than letting the two drift.

Code	Meaning	Retried?
<code>VALIDATION_FAILED</code>	Malformed before any send — empty recipient/subject/body. A programming error.	No
<code>EMAIL_REJECTED</code>	The provider permanently refused — bad recipient, unverified sender, suspended account	No
<code>SERVICE_UNAVAILABLE</code>	Unreachable, timed out, 5xx	Yes
<code>RATE_LIMITED</code>	Send-rate cap hit	Yes

code decides; detail is for humans

`EmailError` carries both. **The code is the whole of the retry decision** and nothing branches on `detail`, which carries the provider's own exception name and message for whoever reads `notification_outbox.last_error` afterwards.

The split exists because collapsing them cost real debugging time: a wrong AWS key surfaced only as `SERVICE_UNAVAILABLE`, indistinguishable from SES being down, while the SDK had already named it `InvalidClientId`. The code was right and the diagnosis was unavailable.

Never render detail to an end user — provider messages quote recipient addresses and account identifiers.

Mapping SES exceptions

`classify` (`ses.ts:107`): named exceptions win, HTTP status is the fallback, everything else is transient.

SES says	Becomes
<code>ThrottlingException</code> , <code>TooManyRequestsException</code> , <code>LimitExceededException</code> , HTTP 429	<code>RATE_LIMITED</code>
<code>MessageRejected</code> , <code>MailFromDomainNotVerifiedException</code> , <code>AccountSuspendedException</code> , <code>SendingPausedException</code> , <code>BadRequestException</code> , HTTP 400	<code>EMAIL_REJECTED</code>
<code>AccessDeniedException</code> , <code>UnrecognizedClientException</code> , <code>InvalidClientId</code> , <code>InvalidSignatureException</code> , <code>SignatureDoesNotMatch</code> , <code>ExpiredTokenException</code> , <code>ExpiredToken</code> , HTTP 403	<code>EMAIL_REJECTED</code>
anything else — network errors, timeouts, 5xx	<code>SERVICE_UNAVAILABLE</code>

403 is **permanent, deliberately**. It means the credentials are wrong, expired, or unauthorised for `ses:SendEmail`, and no amount of retrying makes a bad key good. Treating it as transient hid a missing session token behind eight hours of backoff — see [operations §2](#), which is the same incident from the operator's side.

What the dispatcher does with each outcome

Outcome	Row becomes
ok	sent, with <code>sent_at</code> , <code>provider_message_id</code> , <code>last_error</code> cleared
UNKNOWN_TEMPLATE / INVALID_PAYLOAD	failed — a programming error; retrying replays the bug
EMAIL_REJECTED, VALIDATION_FAILED	failed
SERVICE_UNAVAILABLE, RATE_LIMITED	pending, <code>next_attempt_at</code> = <code>now()</code> + <code>backoff</code>
a thrown error	pending with backoff — an unknown throw is assumed transient , because discarding a notification over a bug in error handling is the worse failure. Capped like the rest.

Retryability is a property of the **failure code**; the cap is a property of the **row**. `MAX_ATTEMPTS = 8` (`dispatch.ts:27`), backoff `min(2 ** attempts, 60)` minutes (`:31`) — doubling, capped at an hour.

Both **retry paths check the cap**, and they must stay symmetric. The typed path at `dispatch.ts:164` tests `TRANSIENT.has(code) && row.attempts < MAX_ATTEMPTS`; the `catch` block at `:128-143` tests `row.attempts < MAX_ATTEMPTS` and calls `fail()` otherwise.

The `catch` branch originally called `retry()` unconditionally, so a send that kept throwing rather than returning an `EmailError` cycled and the row never converged on `failed` while `attempts` climbed past 8. Fixed, with a regression test that fails if the cap check is removed. If you add a third retry path, cap it there too.

A permanent failure logs, and it is the **only** signal that someone was never told:

```
[email:dispatch] permanent failure id=... reason=...
```

Claim before sending

```
update notification_outbox set status = 'sending', attempts = attempts + 1
where id in (
  select id from notification_outbox
  where status = 'pending' and next_attempt_at ≤ now()
  order by created_at
  for update skip locked
  limit $1
)
returning id, template, recipient, cc, payload, attempts
```

`for update skip locked` is the load-bearing clause. Without it a second dispatcher **blocks** on these rows and then sends them again once the first commits. This query is the lock — which is what makes `dispatchOutbox` idempotent and safe to call concurrently.

`attempts` is incremented at **claim** time, not after the send, so a row left in **sending** by a killed process still converges on `MAX_ATTEMPTS` rather than being retried forever.

9 What triggers delivery

Two things, running the same function. You want both.

1. `after()` on every write route. Next's `after` runs work once the response is flushed, so the requestor never waits on SES.

Route	Helper
POST <code>/api/reservations</code>	<code>dispatchAfterResponse</code> — <code>route.ts:70</code>
PATCH <code>/api/reservations/[id]</code>	<code>[id]/route.ts:100</code>
POST <code>/api/reservations/[id]/cancel</code>	<code>[id]/cancel/route.ts:61</code>

Because the claim query takes the oldest due rows regardless of who queued them, a write opportunistically drains anything else pending too.

2. The sweeper — GET `/api/internal/dispatch-outbox`. This is the only thing that fires **when nobody is writing**, which is exactly what you need for mail queued while SES was down. Guarded by an `x-dispatch-secret` header matched against `DISPATCH_SECRET`; an **unconfigured secret returns 404** —

unavailable, not unguarded.

Scheduler-agnostic on purpose: the deployment target is undecided, so it is an ordinary guarded endpoint any cron can call rather than a platform-specific binding. Configuration in [operations §4](#).

Do **not** put the dispatcher in `middleware.ts: /api/*` is outside the matcher by design, and the edge runtime cannot load Kysely.

10 Development tooling

Previews — GET /api/dev/email

An index of 11 fixtures covering every template and every branch worth looking at: single vs. multi-trip, both ride modes, both sites, all three statuses, both cancellers, assigned vs. changed.

- GET /api/dev/email/<name> renders it through the **same render() call the real send uses**, so the browser shows the HTML SES would carry, not an approximation. It sits in an iframe so the page's styles cannot leak in.
- GET /api/dev/email/<name>?format=text returns the plain-text half — the one that usually rots unnoticed — with a **To: / Cc: / Subject:** header block.
- The To/Cc strip resolves from `ADMIN_WHITELIST_SEED` through `selectAdminEmails`, the same function the live path uses. It shows **the rule applied to seed data**, not what any real send did.

Probe — GET /api/dev/email/probe?to=<address>

Sends **one real message** through the app's actual configured transport and reports what the provider said: `emailMode`, `region`, `from`, the access-key **prefix** (`ASIA` vs `AKIA` — never the key), whether a session token is set, and on failure the code plus the provider's own `detail`.

It uses `selectMailTransport(env())` rather than building its own client, so what it proves is the configuration *the app will use*. A probe with its own credentials would answer a question nobody asked.

Both are guarded by `devOnly()` (`preview.ts:9`) — 404 in production. They live under `/api/` because `middleware.ts` excludes `api/`, so a page route at `/dev/email` would be bounced to the login screen before it rendered.

Why “who got this” is answerable in three places

They must agree, and each answers a different question:

Place	Answers
The outbox row's recipient + cc	What a real send actually addressed. Written at enqueue, never recomputed.
StubTransport 's log line	What development would have sent
/api/dev/email 's To/Cc strip	The routing <i>rule</i> , applied to seed data

11 Tests

File	Kind	Covers
<code>service.test.ts</code>	unit	Structural validation, <code>EMAIL_ALWAYS_CC</code> dedup
<code>ses.test.ts</code>	unit	<code>classify</code> over every named exception and status; <code>sesClientConfig</code> credential omission and session-token pass-through
<code>stub.test.ts</code>	unit	The production allowlist throws
<code>select-transport.test.ts</code>	unit	Mode selection and the <code>null</code> cases (4 tests)
<code>recipients.test.ts</code>	unit	<code>selectAdminEmails</code> — filtering, ordering, exclusion, null emails
<code>templates/*.test.ts</code>	unit	Each template's HTML and text; <code>index.test.ts</code> covers schema rejection
<code>notify-payload.test.ts</code>	unit	<code>requestInformationFromDraft</code> only, plus <code>EM_DASH</code> degradation
<code>dispatch.int.test.ts</code>	integration	Permanent vs. transient, backoff, exhaustion on both retry paths, and claim exclusivity under two concurrent dispatchers (14 tests)
<code>db/notification-constraints.int.test.ts</code>	integration	Both CHECK constraints, including <code>notification_outbox_sent_check</code>
<code>recipients.int.test.ts</code>	integration	The rule against the real table
<code>notify.int.test.ts</code>	integration	<code>loadRequestInformation</code> against real rows

Two rules specific to this area:

- **Row cleanup is a hard gate.** Integration tests share one database with `fileParallelism: false`, so a suite that leaks rows breaks a *later* suite while still passing itself. This has happened four times on this project. Any new int test needs an `afterAll` deleting its rows in FK-safe order — `notification_outbox` before `notification_events` before `reservations` — and must pass `pnpm test:int` run twice in a row.

- **dispatchOutbox** takes **alwaysCc** as a parameter (:77) rather than reading the environment per row. `test/global-setup.int.ts` loads `dotenv/config`, so without that seam a real `EMAIL_ALWAYS_CC` in a developer's `.env` would turn a cc assertion red for a reason that has nothing to do with dispatching.

12 Known gaps and things that will bite

#	Issue	Consequence
1	The in-app channel has no read path. <code>notification_events</code> rows accumulate; nothing selects them, and <code>read_at</code> is never set.	The bell is unbuilt. Rows are correct and ready; the feature is not shipped.
2	<code>sendBookingStatusChange</code> (<code>service.ts:101</code>) is dead. Only its own test calls it — the live path goes through <code>renderTemplate</code> + <code>sendEmail</code> .	Dead production code. Safe to delete once nothing claims it.
3	<code>tripDetails</code> and <code>driver.carType</code> are loosened for backwards compatibility (<code>templates/index.ts:42, :54</code>).	Past their release, a genuinely missing field renders as a dash instead of failing loudly. Tighten once no pre-deploy row can remain.
4	A stub built in production leaves rows pending forever, silently. <code>StubTransport</code> 's constructor throws inside <code>selectMailTransport</code> (<code>select-transport.ts:37</code>), which every caller runs <i>before</i> <code>dispatchOutbox</code> , so the dispatcher never starts and rows just stay <code>pending</code> . That is the right outcome and nothing alerts you to it .	Watch the count of <code>pending</code> rows older than an hour — operations \$5 .
5	An empty rendered field fails the row permanently, with no mail and no exception. <code>sendEmail</code> returns <code>VALIDATION_FAILED</code> for an empty recipient, subject or body.	This is why every payload string is <code>min(1)</code> and why missing values become <code>EM_DASH</code> . Validate payload shape at enqueue time.
6	Forward-looking, not current: <code>env.ts</code> warns that a shell-exported <code>AWS_*</code> pair is picked up by the SDK's default credential chain "even when <code>EMAIL_MODE=stub</code> ". That is a property of the <i>variable names</i> , not of this pipeline — <code>@aws-sdk</code> is imported only by <code>ses.ts</code> , and <code>new SESv2Client</code> runs only inside <code>SesTransport</code> , which exists only under <code>EMAIL_MODE=ses</code> .	Nothing to watch today. It becomes real the moment any other AWS client is added to the app.
7	The submitted event's null <code>reservation_id</code> escapes ON DELETE CASCADE.	Harmless today; a future hard-delete would strand those rows.

#	Issue	Consequence
8	Nothing alerts on a permanent failure. The only signal is a <code>console.error</code> line.	If logs don't ship somewhere with alerting, a requestor who was never told is invisible.

See also

- [Domain §9](#) — who is notified, as a business rule
- [Operations §2](#) — SES configuration and the session-token trap
- [Operations §4](#) — running and configuring the sweeper
- [Architecture §6](#) — the full table and migration inventory
- [Local setup](#) — getting mail working locally